Read-Write Locks (RW Locks)

What are Read-Write Locks?

Read-Write Locks are synchronization mechanisms that allow:

Multiple Readers
OR
Single Writer

at a time.

They improve performance when data is read frequently and written rarely.

Why Do We Need RW Locks?

Suppose a shared resource is accessed as follows:

95% Reads
5% Writes

Using a normal mutex/spinlock:

Reader1 -> Read
Reader2 -> Wait
Reader3 -> Wait

Even though readers are only reading data, they block each other.

RW Locks allow multiple readers to access data simultaneously.

Basic Principle

Multiple Readers Allowed

Reader1 -> Reading
Reader2 -> Reading
Reader3 -> Reading

All readers can access data concurrently.

Single Writer Allowed

Writer1 -> Writing

Writer2 -> Waiting

Only one writer can modify data.

Readers and Writers Cannot Execute Together

Writer -> Writing

Reader1 -> Waiting
Reader2 -> Waiting

OR

Reader1 -> Reading
Reader2 -> Reading

Writer -> Waiting

Types of Read-Write Locks

Linux provides two types:

1. RW Spinlock
2. RW Semaphore

1. RW Spinlock

Header File:

#include <linux/rwlock.h>

Declaration:

rwlock_t rwlock;

Characteristics:

Busy Waiting
No Sleeping
Short Critical Sections
Interrupt Context Supported

APIs

Initialization

rwlock_init(&rwlock);

Reader Lock

read_lock(&rwlock);

/* Read Critical Section */

read_unlock(&rwlock);

Writer Lock

write_lock(&rwlock);

/* Write Critical Section */

write_unlock(&rwlock);

2. RW Semaphore

Header File:

#include <linux/rwsem.h>

Declaration:

struct rw_semaphore rwsem;

Characteristics:

Sleeping Lock
Long Critical Sections
Process Context Only

APIs

Initialization

init_rwsem(&rwsem);

Reader Lock

down_read(&rwsem);

/* Read Critical Section */

up_read(&rwsem);

Writer Lock

down_write(&rwsem);

/* Write Critical Section */

up_write(&rwsem);

RW Spinlock vs RW Semaphore

RW Spinlock	RW Semaphore
Busy Wait	Sleep
No Sleeping Allowed	Sleeping Allowed
Short Critical Section	Long Critical Section
Interrupt Context	Process Context
Faster	Higher Overhead

Advantages

Concurrent Readers

Reader1
Reader2
Reader3
Reader4

All readers can execute simultaneously.

Better Performance

Suitable when:

Many Reads
Few Writes

Improved Scalability

Multiple CPUs can read shared data concurrently.

Disadvantages

Writer Starvation

If readers continuously access the lock:

Reader1
Reader2
Reader3
Reader4
...

Writer may wait indefinitely.

Writer -> Waiting

This is called:

Writer Starvation

When to Use RW Locks?

Use RW Locks when:

✓ Read operations are frequent
✓ Write operations are rare
✓ Multiple readers need simultaneous access
✓ Read performance is important

Examples:

Device Configuration
Routing Tables
Kernel Statistics
Status Information
Lookup Tables

When Not to Use RW Locks?

Avoid RW Locks when:

Reads ≈ Writes

or

Mostly Writes

In such cases:

Mutex
Spinlock

are generally better choices.

Why use RW Locks?

To allow multiple readers to access shared data concurrently while still ensuring exclusive access for writers.

Difference Between Read Lock and Write Lock?

Read Lock:

Multiple readers allowed

Write Lock:

Only one writer allowed
No readers allowed

What is Writer Starvation?

A situation where continuous readers prevent a waiting writer from acquiring the lock.

Types
---
rwlock_t -> Spin Based
rw_semaphore -> Sleep Based

APIs
---
read_lock()
read_unlock()

write_lock()
write_unlock()

down_read()
up_read()

down_write()
up_write()

Problem

Writer Starvation

Easy Memory Trick

Many Reads + Few Writes
↓
RW Lock

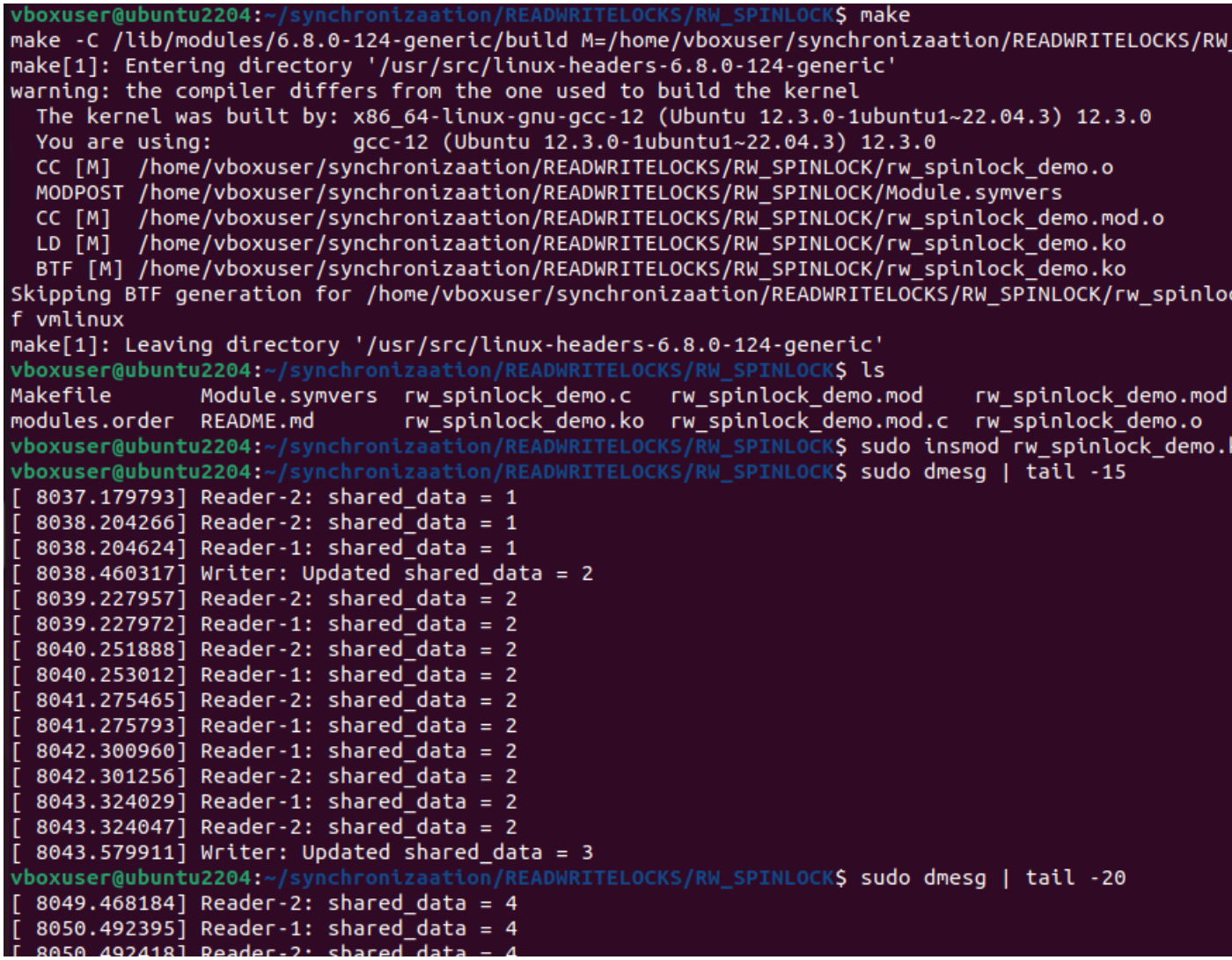
Short Critical Section
↓
RW Spinlock

Long Critical Section

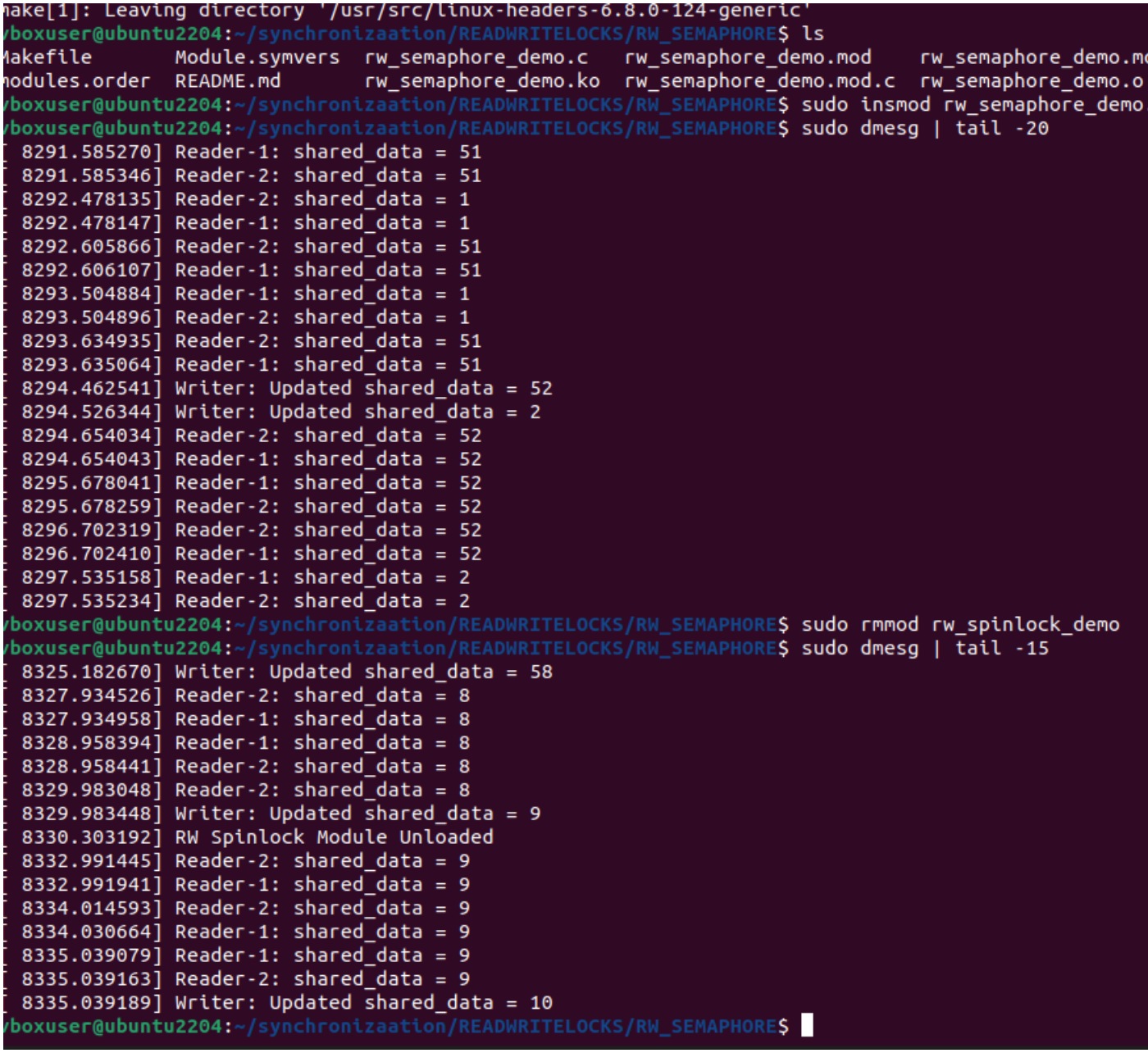
↓

RW Semaphore

RW-SPINLOCK OUTPUT:



RW-SEMAPHORE OUTPUT:



Q: Can a read lock be upgraded directly to a write lock?

Answer:

No. Linux Read-Write locks do not support automatic lock upgrading. If a thread holding a read lock tries to acquire a write lock, it will deadlock because the write lock waits for all readers to release the lock, including the current thread.

downgrade write()

Read writer semaphores have a unique method which is not present in reader-writer spinlocks.

This function automatically convert an acquired write lock to a read lock

Use case: used in the situation where writer lock is needed for a quick change, followed by longer period of read -only access.